→ Queue :-

A Queue is a Linear data structure that works on the FIFO principle:- First In, First Out. It's like a line of people at a ticket counter: the person who comes first get served first.

→ Basic operations:

(i) enqueue() → Adds an element to the rear of the queue
(ii) dequeue() → Removes an element from the front of the queue
(iii) front() → Returns the first element without removing it.
(iv) rear() → Returns the last element.
(v) isEmpty() → checks if the queue is empty.
(vi) isFull() → checks if the queue is full (for static queues)

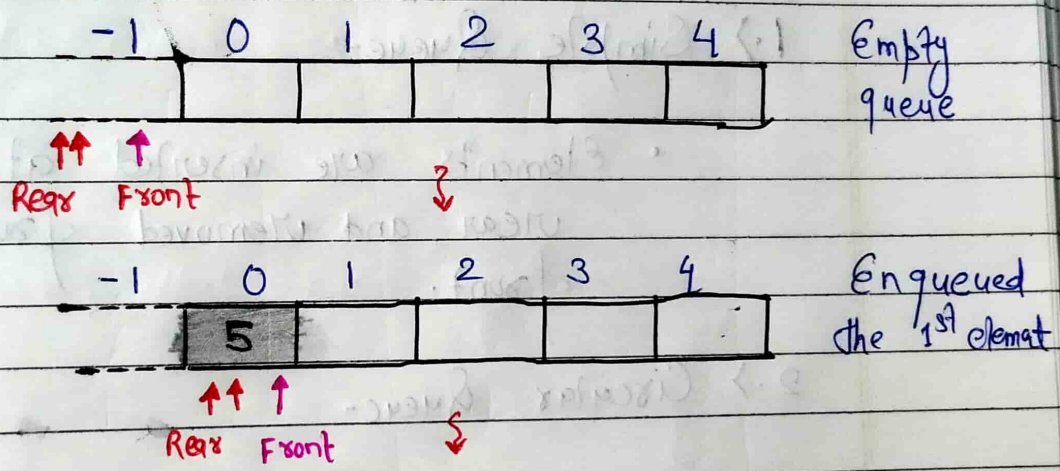
Front → In a queue, the front is the position where element are removed (dequeued) - It always points to the first element that was inserted.

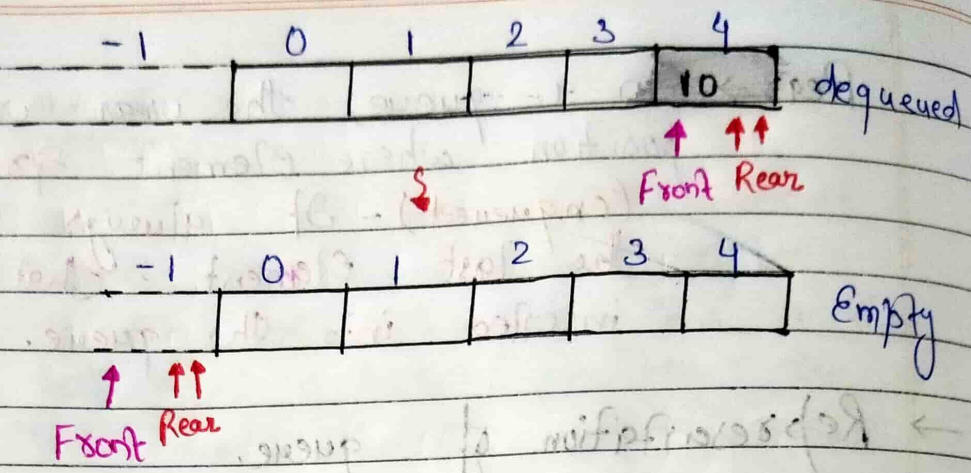
Akash

Rear → In a queue, the rear refers to the position where element are added (enqueued) - It always point to the last element that was inserted into the queue.

→ Representation of queue.

Front = -1
Rear = -1





→ Types of Queues:

1.) Simple Queue-

- Elements are inserted at the rear and removed from the front.

2.) Circular Queue-

- The last position is connected back to the first position.

3.) Priority Queue

- Each element has a priority
- Elements are dequeued of wasted space in linear queue.

4.) Double-Ended Queue (Deque)

- Insertion and deletion are possible from both the front and rear.

Akash

→ Algorithm to create an queue using array implementation.

Step 1: set max 100

Step 2: set front = -1

Step 3: set rear = -1

Step 4: IF (n > max)
print "queue overflow"

Step 5: else,
IF (n) = 0)
front ++

Step 6: Under for loop condition

6.1. queue[++rear] = val

Step 7. Exit.

Akash

→ Algorithm to display queue:

Step 1: IF (front == -1)
print "queue is empty"

Step 2: else,

2.1. Under for loop condition
print queue [i];

Step 3: Exit.

Akash

→ // C program to create & display quere

```
#include<stdio.h>
#define max 100

int queue[max], front=-1, rear=-1;
void create_queue()
{
    int i, n, val;
    printf("enter size of queue \n");
    scanf("%d", &[redacted]n);

    if (n>max)
    {
        printf("queue overflow");
    }
    else
    {
        if(n != 0)
        {
            front++;
        }
        for(i=0; i<n; i++)
        {
            printf("enter data");
            scanf("%d", &val);
            queue[++rear]=val;
        }
    }
}
```

Akash

```
void display()
{
    int i;
    if(front == -1)
    {
        printf("queue is empty");
    }
    else
    {
        printf("queue elements:\n");
        for(i=0; i<=rear; i++)
        {
            printf("%d\n", queue[i]);
        }
    }
}
int main()
{
    Create_queue();
    display();
    return 0;
}
```

Akash